

Threat Intelligence Correlation Platform

1. First Understand the Problem Statement

Your problem statement is:

Defence analysts receive thousands of alerts every day from SIEM systems, cyber sensors, OSINT feeds, and classified intelligence reports. These alerts come in different formats. Humans cannot read and analyze all of them. Missing a genuine threat is extremely dangerous, while investigating false positives wastes analysts' time. When a genuine threat is detected, a structured threat assessment in BLUF format must be produced within minutes so commanders can make decisions within their operational window.

In simple words:

The Current Situation

Imagine an organization receives this in one day:

SIEM	→ 8,000 alerts
Cyber Sensors	→ 5,000 events
OSINT	→ 1,000 reports
Intelligence Reports	→ 300 documents
	14,300+ items

Now imagine a human analyst has to read these and determine:

"Is there an actual threat hidden somewhere in these 14,300 pieces of information?"

That's the core problem.

2. What Is the Actual Problem?

There are actually **5 problems** hidden inside the statement.

Problem 1 — Too Much Data

Analysts receive huge volumes of information.

```
Thousands of alerts
    ↓
Human analysts
    ↓
Cannot manually read everything
```

This creates **information overload**.

Problem 2 — Different Formats

The information doesn't arrive in one standard format.

For example:

```
SIEM
{
  "source_ip": "...",
  "severity": "high"
}

OSINT
"Reports indicate suspicious activity..."

Intelligence PDF
"On 5 October, activity associated with..."
```

So the system has to understand:

JSON + logs + text + PDF + reports + structured alerts

and bring them into a common format.

3. Problem 3 — False Positives

Suppose there are 10,000 alerts.

Maybe:

```
10,000 alerts
    ↓
```

```
9,500 normal/irrelevant
↓
500 suspicious
↓
20 serious
```

If analysts investigate all 10,000 manually, enormous time is wasted.

This is the **false-positive problem**.

The system should therefore answer:

Which alerts actually deserve analyst attention?

4. Problem 4 — Correlation

This is probably the **most important word in your problem statement**.

Consider:

```
SIEM:
Suspicious IP = 192.xxx.xxx.xxx

+

Cyber Sensor:
Same IP contacted internal server

+

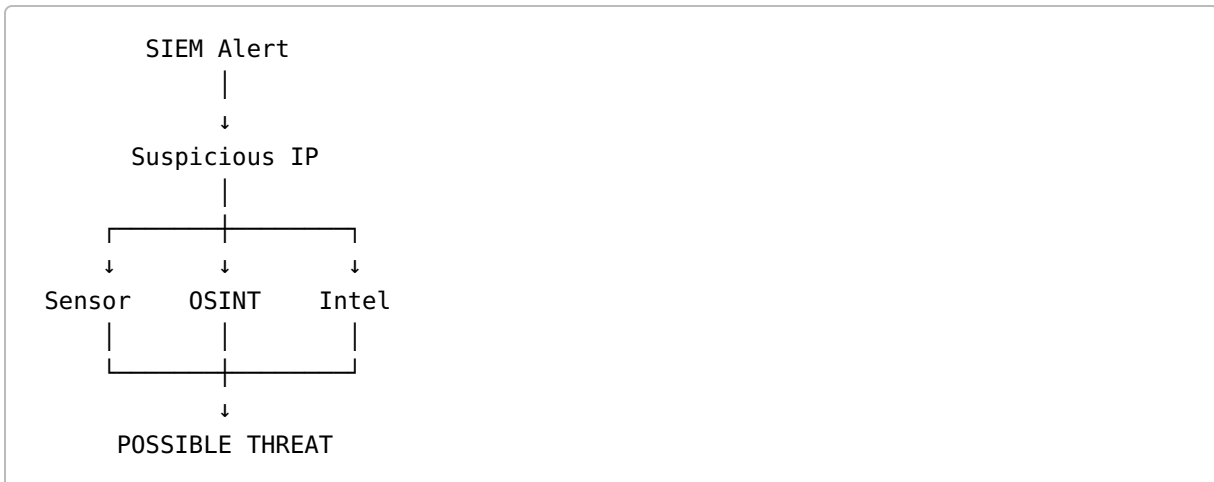
OSINT:
IP associated with known threat actor

+

Intelligence Report:
Threat actor targeting defence organizations
```

Individually, these might look like four separate pieces of information.

But together:



This is **Threat Intelligence Correlation**.

Your system should discover these relationships automatically.

5. Problem 5 — Analysts Need an Answer Quickly

Even after discovering a threat, analysts may have to manually prepare an intelligence assessment.

The problem says:

BLUF within minutes, not hours.

BLUF = **B**ottom **L**ine **U**p **F**ront.

Instead of giving the commander 50 pages of raw information:

```
50 pages of reports
  ↓
Commander
  ↓
"What is happening?"
"What is the threat?"
"How serious is it?"
"What should we do?"
```

Your system should produce something like:

BLUF

Threat: Potential coordinated cyber activity

Confidence: HIGH

Affected Target: Defence infrastructure

Evidence:

- 12 SIEM alerts
- 4 sensor events
- 2 OSINT reports
- 1 intelligence report

Assessment:

Multiple independent sources indicate potential coordinated activity.

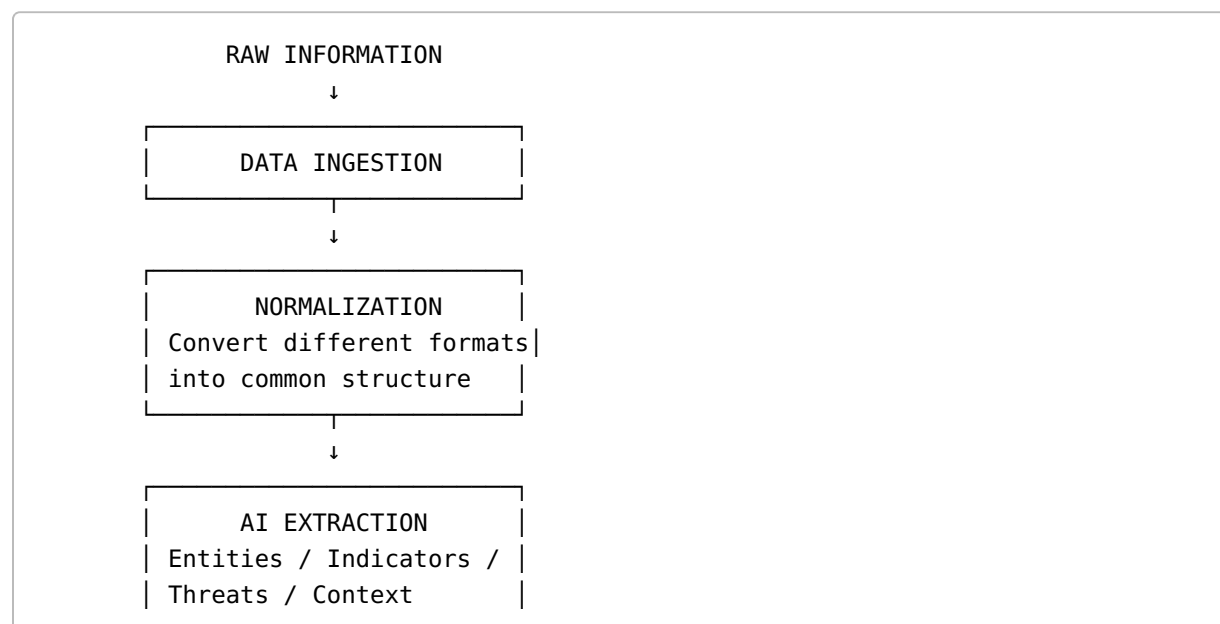
Priority: CRITICAL

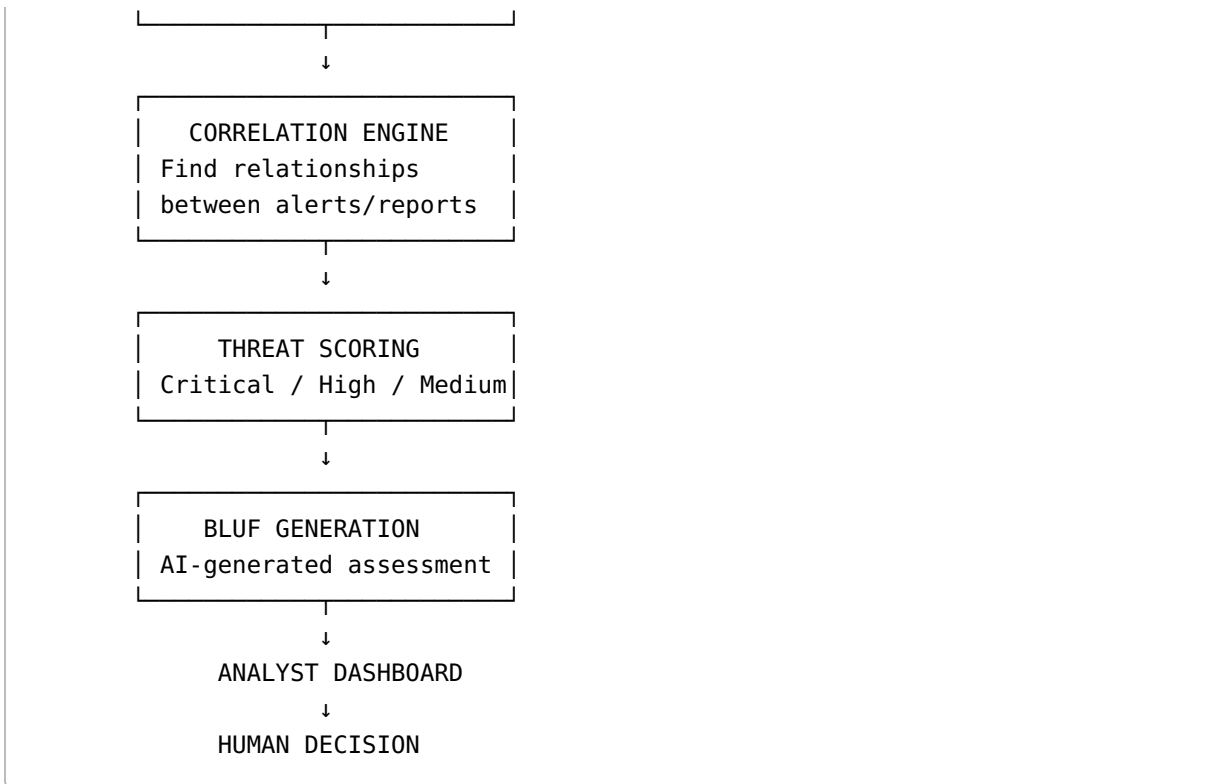
Analyst Review: REQUIRED

That's the **decision-support part** of your system.

6. So What Does Your Software Actually Need to Do?

Your project can be thought of as:





That is the **technical overview** you should keep in your head.

7. What Does "AI" Actually Do?

Don't make the mistake of saying:

"We'll put an LLM and it will detect threats."

That's too vague.

You need several AI/technical components.

AI/NLP Can Do:

1. Classification

Determine what type of information something is.

Alert → Malware activity
Report → Threat actor activity
Log → Network anomaly

2. Entity Extraction

Extract important entities:

IP
Domain
Hash
Threat Actor
Malware
Organization
Location
Timestamp

3. Summarization

Convert long reports into short summaries.

4. Semantic Similarity

Determine whether two reports are talking about similar activity even when the wording is different.

5. Threat Assessment

Combine evidence and produce a structured assessment.

8. But AI Alone Isn't Enough

Your system needs a combination of:

AI/LLM
+
Rules
+
Database
+
Search
+

```
Correlation
+
Scoring
+
Human Analyst
```

For example:

AI says:

These two reports appear related.

Rule Engine says:

Both mention the same IP.

Database says:

This IP appeared in 17 previous alerts.

Threat Intelligence says:

This indicator has previously been associated with a known threat actor.

Then your system can produce:

High-confidence correlated incident

This is much stronger than asking an LLM to make the decision alone.

9. Your Minimum Viable Product for the Hackathon

Since you have only a few days, **don't try to build a real military-grade system.**

Build a convincing **prototype.**

I'd recommend:

Input

Allow the user to upload:

- PDF
- TXT
- JSON
- CSV

and/or generate simulated SIEM alerts.

Processing

```
Upload
↓
Extract text/data
↓
Normalize
↓
AI entity extraction
↓
Find similar events
↓
Correlate
↓
Threat score
↓
Generate BLUF
```

Output

A dashboard showing:

Total Alerts: 1,250

Critical: 8

High: 31

Medium: 182

Low: 1,029

TOP CORRELATED THREATS

● Threat Cluster #17

Confidence: 94%

Related Sources:

SIEM

OSINT

Sensor

Intel Report

Indicators:

IP: xxx.xxx.xxx.xxx

Domain: example.xxx

Actor: Unknown

[View BLUF]

Then clicking **View BLUF** shows the generated assessment.

10. The Most Important Demo Scenario

For the hackathon, create a **synthetic dataset**.

Suppose you feed the system:

File 1 — SIEM

Suspicious connection from IP X

File 2 — Cyber Sensor

Internal server contacted IP X

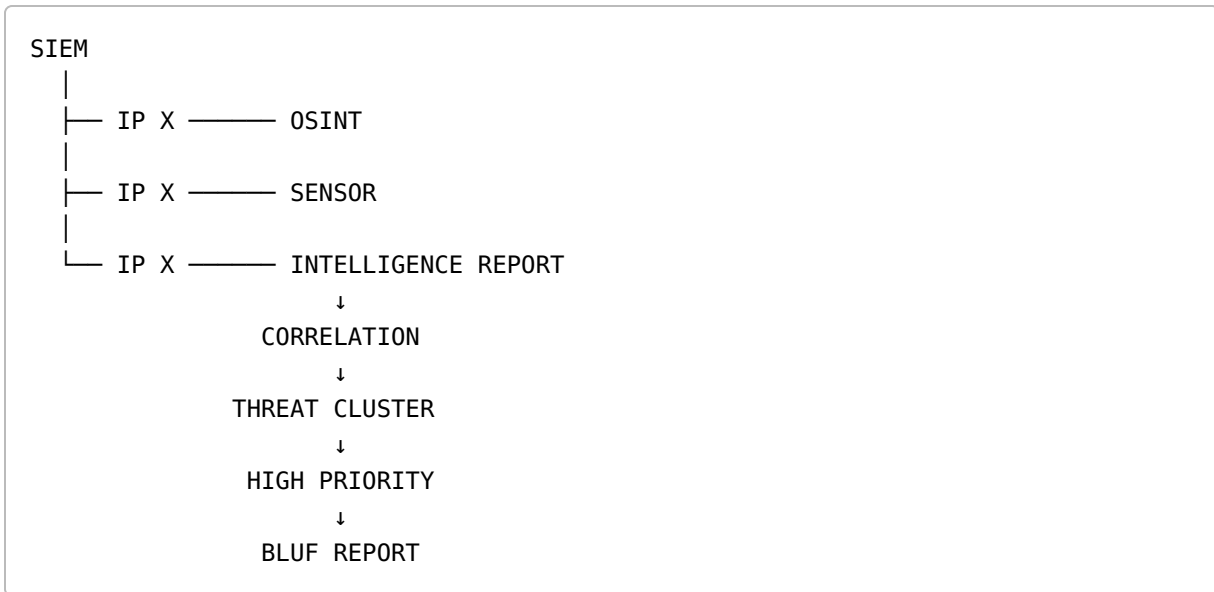
File 3 — OSINT

IP X associated with suspicious infrastructure

File 4 — Intelligence Report

Threat actor Y has previously used infrastructure associated with IP X.

Your system should automatically discover:



That single demo will communicate your entire idea to the judges.

11. Technical Stack I'd Recommend for You

Since you already know Java/Spring Boot and React:

Frontend

React + TypeScript + Tailwind

Dashboard, alerts, threat clusters, BLUF reports.

Backend

Spring Boot

Handles:

- File upload
- APIs
- Authentication
- Alert management
- Threat scoring
- Report management

Database

PostgreSQL

Store:

Alerts
Reports
Indicators
Threat Actors
Threat Clusters
Analyst Feedback
BLUF Reports

AI

For the hackathon, you could use:

Ollama + local LLM if you want local/private processing.

Or an API-based LLM if your hackathon permits it.

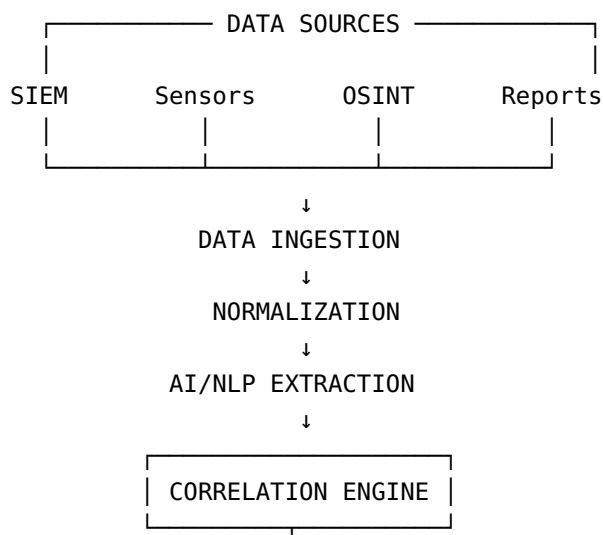
Search / Similarity

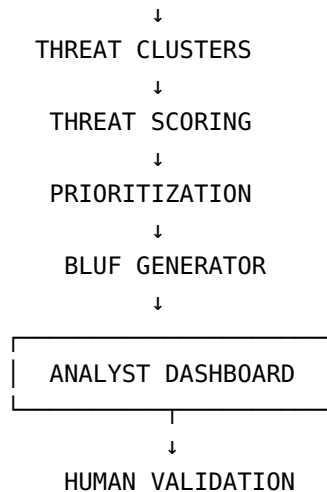
Use:

pgvector with PostgreSQL

for semantic similarity between reports/alerts.

12. Your Final System in One Picture





In One Sentence

Your project is basically:

"A platform that converts thousands of heterogeneous intelligence alerts and reports into a small number of correlated, prioritized threat incidents and automatically produces evidence-backed BLUF assessments for analysts."

What You Should Focus on First

For **9 October**, don't start with UI or advanced AI. Your first technical milestone should be:

Different inputs → common format → identify entities → correlate related information → calculate priority → generate BLUF.

Once this pipeline works, the dashboard is comparatively easy to build.